

PCAFondo.com

Resumen técnico de la tienda online — Julio 2026

1. Introducción

PCAFondo.com es una tienda online de tecnología y electrónica construida con **Next.js 16** + **Supabase**. Los clientes pueden navegar productos, ver detalles, agregar al carrito y hacer pedidos. El administrador (usuario con rol admin) gestiona productos, pedidos y el dashboard desde un panel privado.

Tecnología	Versión	Uso
Next.js	16.2.9	Framework React (App Router, SSR, API Routes)
React	19.2.4	UI interactiva del lado cliente
Supabase	^0.12.0 / ^2.108.2	Auth, base de datos PostgreSQL, almacenamiento de imágenes
CSS	—	Globals.css + Font Awesome + Google Fonts (Orbitron, Roboto)

2. Arquitectura general

2.1 Capas

Capa	Responsabilidad	Tecnología
Presentación	Renderizar páginas, manejar interacciones	Server/Client Components
Estado cliente	Auth + Carrito en el browser	React Context (auth-context, cart-context)
API Routes	Operaciones sensibles (admin, upload)	Next.js Route Handlers + Supabase server client
Middleware	Refrescar sesión, proteger rutas	proxy.js → updateSession()
Base de datos	Datos de productos, pedidos, usuarios	Supabase PostgreSQL + RLS policies
Storage	Imágenes de productos	Supabase Storage (bucket products)

2.2 Estructura de archivos clave

```
app/  
  layout.js - Raíz (AuthProvider, CartProvider, Header, Footer, CartSidebar, Toast)  
  page.js - Home (carousel + grilla de productos + filtros)  
  checkout/page.js - Checkout (formulario + crear pedido)  
  mis-pedidos/page.js - Historial del cliente  
  producto/[slug]/page.js - Detalle producto (server + client)
```

admin/layout.js - Sidebar admin + protección de ruta
 admin/page.js - Dashboard con estadísticas
 admin/productos/page.js - CRUD de productos
 admin/pedidos/page.js - Gestión de pedidos (cambiar estado)
 api/admin/products/route.js - PUT (toggle) + DELETE producto
 api/upload/route.js - Subir imágenes a Storage
 auth/callback/route.js - Callback OAuth

lib/

supabase/client.js - Cliente browser (anon key)
 supabase/server.js - Cliente server (anon key + cookies)
 supabase/middleware.js - Lógica de refresh de sesión
 supabase/queries.js - Helpers de consultas server-side
 context/auth-context.js - Contexto de autenticación
 context/cart-context.js - Contexto del carrito (persistido en localStorage)

components/

header.js / footer.js / cart-sidebar.js / product-card.js / login-modal.js / toast.js / carousel.js / benefits.js

3. Páginas y funcionalidades

Ruta	Qué hace	Tipo	Autenticación
/	Home: carrusel promocional, grilla de productos con filtros (todos/en stock/a pedido), sección de beneficios. Los datos se obtienen server-side.	Server	Pública
/producto/[slug]	Detalle del producto: imágenes (galería), precio, descripción, especificaciones, selector de cantidad, botón "Agregar al Carrito". SEO con metadata dinámica.	Server+Client	Pública
/checkout	Finalizar compra: formulario con datos de envío, método de pago (transferencia/Mercado Pago), resumen del carrito. Crea la orden en DB.	Client	Usuario logueado
/mis-pedidos	Historial de pedidos del cliente con estados (pendiente, pagado, preparando, enviado, entregado, cancelado).	Client	Usuario logueado
/admin	Dashboard con tarjetas de: cantidad de productos, pedidos totales, pedidos pendientes, ingresos.	Client	Admin
/admin/productos	Lista de productos con toggle activar/pausar y botón eliminar. Usa API route para las mutaciones.	Client	Admin
/admin/productos/nuevo	Formulario para crear producto: nombre, slug, precio, descuento, stock, categoría, especificaciones, imágenes, destacado.	Client	Admin
/admin/pedidos	Lista de pedidos con selector para cambiar estado. Muestra datos del cliente y productos.	Client	Admin

4. Flujo de datos

4.1 Navegación pública (productos)

Browser → solicita página → **Next.js Server** ejecuta el Server Component
→ **lib/supabase/server.js** (cliente server con anon key + cookies)
→ **Supabase** consulta SELECT en tabla products
→ RLS policy select_all permite ver todos los productos
→ Server Component renderiza HTML → se envía al browser

4.2 Autenticación

Login/Register → **LoginModal** llama a `supabase.auth.signInWithPassword()` o `signUp()`
→ Supabase Auth devuelve sesión → se guarda en cookies (sb-*)
→ **proxy.js** (middleware) refresca la sesión en cada request
→ **AuthContext** detecta el cambio → carga el **profile** desde tabla profiles
→ Los componentes reaccionan: Header muestra nombre, AdminLayout verifica rol admin

4.3 Operaciones admin (toggle/delete producto)

Admin page (browser) → `fetch('/api/admin/products')`
→ **API Route** crea Supabase server client (lee cookies de la request)
→ **checkAdmin()** verifica: `auth.getUser() + profiles.role = 'admin'`
→ Si es admin → ejecuta `update()` o `delete()` en Supabase
→ RLS policy update_auth / delete_auth permite porque el usuario está autenticado vía cookies
→ Respuesta JSON → Admin page actualiza el estado local

4.4 Checkout (crear pedido)

Checkout page (browser) → usuario completa formulario y confirma
→ `supabase.from('orders').insert()` — browser directo con anon key
→ RLS policy permite INSERT si usuario está autenticado
→ Luego inserta `order_items` con los productos del carrito
→ **CartContext** ejecuta `clearCart()` → redirige a /mis-pedidos

4.5 Carrito (estado local)

CartContext (React Context) mantiene los items en memoria
→ Persiste en `localStorage` (clave 'cart')
→ Se recupera al cargar la página (efecto en mount)
→ Funciones: `addItem`, `removeItem`, `updateQty`, `clearCart`
→ Los componentes **Header** y **CartSidebar** consumen el contexto

5. Base de datos

5.1 Tablas

Tabla	Columnas principales	Propósito
profiles	id (UUID, FK auth.users), name, role (admin customer), created_at	Perfil de usuario, vinculado a auth.users
products	id (UUID), name, slug, description, price, compare_price, stock_type, stock_quantity, category, images (JSONB), specs (JSONB), featured, active, deleted, created_at	Catálogo de productos
orders	id (UUID), user_id, total, status, shipping_name, shipping_address, shipping_phone, payment_method, notes, created_at	Pedidos de clientes
order_items	id (UUID), order_id (FK orders), product_id, product_name, quantity, unit_price	Items de cada pedido

5.2 RLS Políticas (products)

Operación	Policy	¿Qué permite?
SELECT	select_all	Cualquier persona (público)
INSERT	insert_auth	Usuarios autenticados Admin check en app
UPDATE	update_auth	Usuarios autenticados Admin check en app
DELETE	delete_auth	Usuarios autenticados Admin check en app

Las policies están abiertas para autenticados porque la verificación de rol admin se hace en las API routes (checkAdmin). Esto evita problemas con subconsultas RLS recursivas mientras se mantiene la seguridad a nivel de aplicación.

6. Seguridad

Capa	Mecanismo	Detalle
Middleware	proxy.js	Refresca sesión en cada request. Redirige a / si el usuario no está autenticado en rutas protegidas (/admin, /checkout, /mis-pedidos). Verifica rol admin para rutas /admin/*.
API Routes	checkAdmin()	Toda operación de escritura sensible (admin products, upload) verifica que el usuario esté autenticado y tenga rol admin en la tabla profiles.
RLS	Row-Level Security	SELECT público para productos. INSERT/UPDATE/DELETE restringido a usuarios autenticados. Doble capa: RLS + verificación en código.
Auth	Supabase Auth	Autenticación por email/contraseña. Sesión manejada con cookies HttpOnly vía @supabase/ssr.
Storage	API route + server client	Subida de imágenes solo a través de API route que verifica admin. No se expone el bucket directamente al browser.

7. Mapa del sitio (navegación)

```
/ (Home – pública)
├─ /producto/[slug] (Detalle producto – pública)
├─ /checkout (Finalizar compra – requiere login)
├─ /mis-pedidos (Historial – requiere login)
└─ /admin (Panel admin – requiere rol admin)
    ├─ /admin (Dashboard)
    ├─ /admin/productos (Lista con toggle/delete)
    │   ├─ /admin/productos/nuevo (Crear)
    │   └─ /admin/productos/editar/[id] (Editar)
    └─ /admin/pedidos (Gestionar pedidos)
```

8. Observaciones técnicas

- **Middleware en Next.js 16:** se llama `proxy.js` en vez de `middleware.js` (cambio de convención en esta versión).
- **Carrito en `localStorage`:** No persiste en DB. Al cerrar sesión o cambiar de dispositivo se pierde. Es intencional para simplicidad.
- **Stock:** El checkout actualmente no valida stock disponible. Se descuenta solo cuando se crea el pedido. No hay control de disponibilidad en tiempo real.
- **Pagos:** Mercado Pago y Transferencia Bancaria son placeholders — no hay integración real de pasarela de pagos. El método de pago se guarda como texto.
- **Imágenes:** Se suben a Supabase Storage (bucket `products`) mediante API route. No hay límite de tamaño ni compresión.
- **Estados de pedido:** `pending` → `paid` → `preparing` → `shipped` → `delivered` (o `cancelled`). El admin los cambia manualmente desde el panel.
- **No hay test:** El proyecto no incluye tests automatizados de momento.

Documento generado el `${new Date().toLocaleDateString('es-AR')}` — PCAFondo.com / mi-proyecto